



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2
із дисципліни «Технології розроблення програмного забезпечення»
Тема: «Основи проектування»

Виконала:
студентка групи ІА-31:
Мартишенко І.С.

Перевірив:
Мягкий М.Ю.

Тема: Основи проектування.

Мета: Обрати зручну систему побудови UML-діаграм та навчитися будувати діаграми варіантів використання для системи що проектується, розробляти сценарії варіантів використання та будувати діаграми класів предметної області.

Посилання на репозиторій з проектом:

https://github.com/martyshenkoo/music_player

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Проаналізувати тему та спроектувати діаграму варіантів використання відповідно до обраної теми лабораторного циклу.
- Спроектувати діаграму класів предметної області.
- Вибрати 3 варіанти використання та написати за ними сценарії використання.
- На основі спроектованої діаграми класів предметної області розробити основні класи та структуру бази даних системи. Класи даних повинні реалізувати шаблон Repository для взаємодії з базою даних.
- Нарисувати діаграму класів для реалізованої частини системи.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму варіантів використання відповідно, діаграму класів системи, вихідні коди класів системи, а також зображення структури бази даних.

Хід роботи

Діаграма варіантів використання:

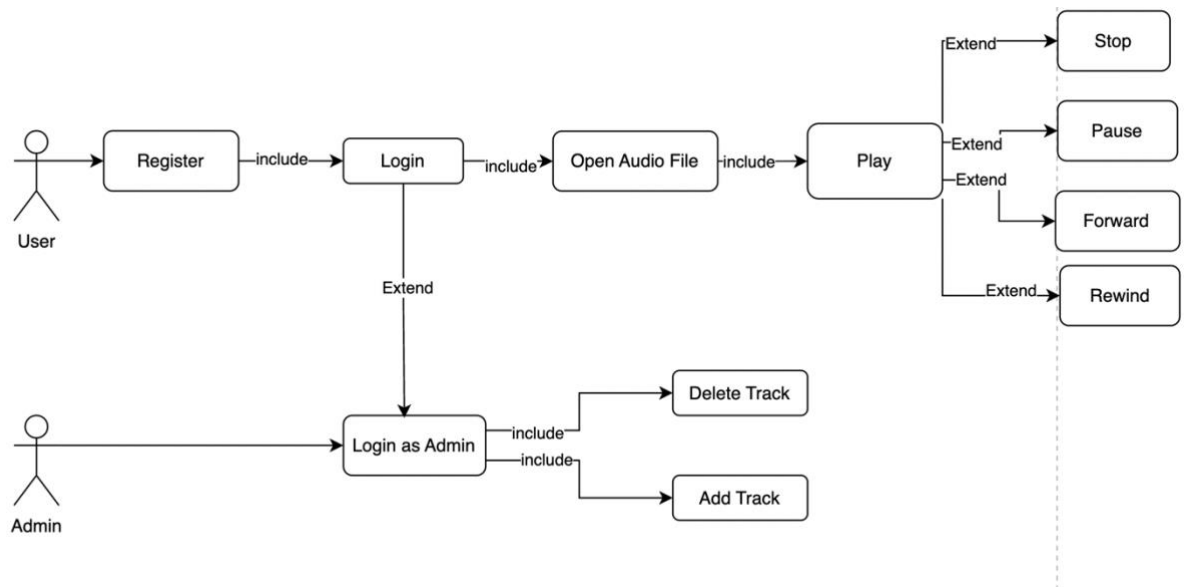
Опис:

Актор - **Користувач** - взаємодіє з системою та має доступ до таких функцій:

- Зареєструватися
- Увійти
- Відкрити аудіофайл
- Відтворити музику
- Поставити на паузу
- Зупинити відтворення
- Перемотати вперед
- Перемотати назад
- Вийти з програми

Актор - **Адмін** - має всі функції користувача, а також додаткові можливості адміністрування:

- Увійти як адмін
- Додати трек
- Видалити трек



Діаграма варіантів використання

Діаграма класів предметної області:

Клас **User** моделює звичайного користувача музичного програвача. Він відповідає за базову взаємодію з системою: реєстрацію, авторизацію та керування відтворенням.

Атрибути:

- id: int - унікальний ідентифікатор користувача
- username: String - логін користувача
- password: String - пароль користувача

Операції:

- login(): авторизація в системі
- logout(): вихід із системи

- `openFile()`: відкриття аудіофайлу
- `playTrack()`: відтворення треку
- `pause()`: призупинення відтворення
- `stop()`: зупинка відтворення
- `rewind()`: перемотування назад
- `forward()`: перемотування вперед

Клас `Admin` є розширенням користувача і моделює адміністратора системи. Він має додаткові можливості для керування аудіоконтентом.

Наслідує: `User`

Операції:

- `addTrack(t: Track)`: додавання нового треку
- `deleteTrack(t: Track)`: видалення треку
- `loginAsAdmin()`: вхід із правами адміністратора

Клас `Player` моделює поведінку музичного програвача. Він керує поточним треком і виконує команди відтворення.

Атрибути:

- `currentTrack`: `Track` - поточний трек, який програвється

Операції:

- `play()`: почати відтворення
- `pause()`: пауза
- `stop()`: зупинити трек
- `rewind()`: перемотати назад
- `forward()`: перемотати вперед

Клас `Track` моделює окремий музичний трек. Він містить основну інформацію про аудіофайл.

Атрибути:

- `id`: `int` - унікальний ідентифікатор треку

- title: String - назва треку
- artist: String - ім'я виконавця
- duration: int - тривалість у секундах

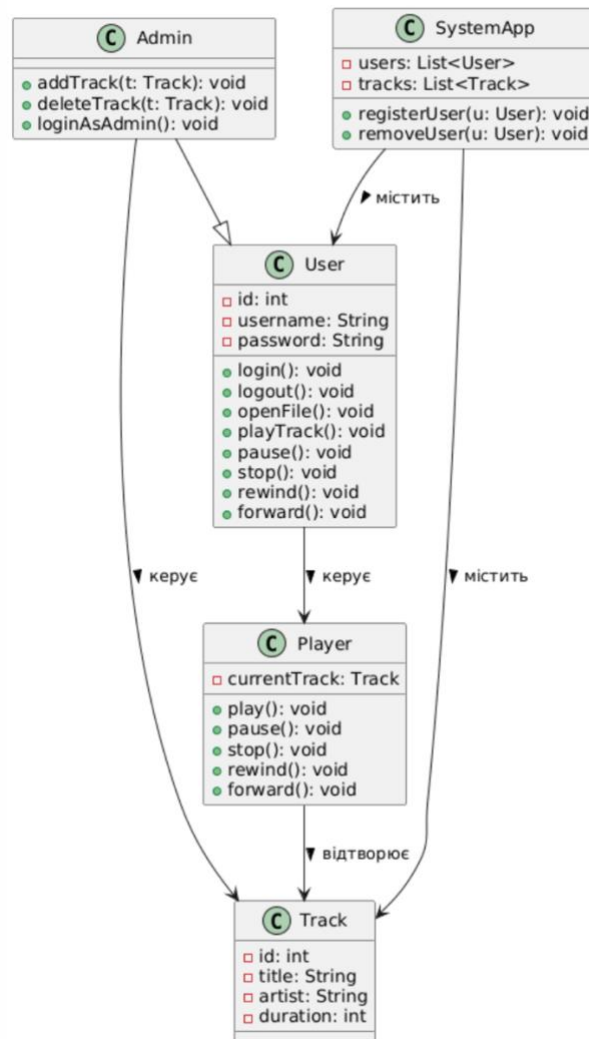
Клас **SystemApp** моделює всю систему програвача. Він зберігає колекцію користувачів та треків і відповідає за реєстрацію та видалення користувачів.

Атрибути:

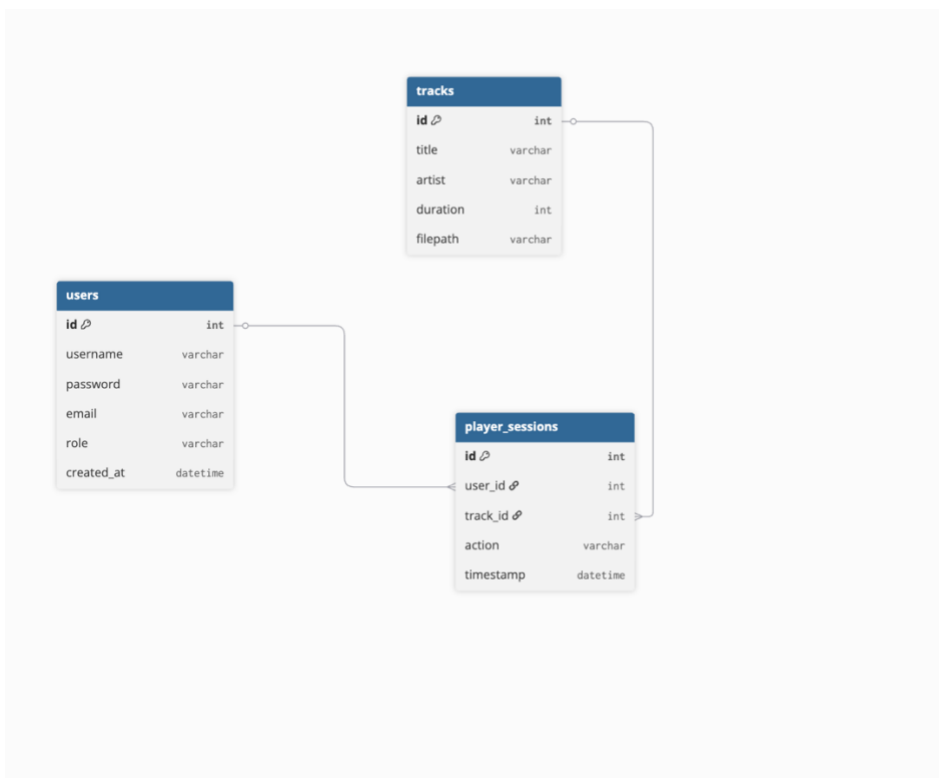
- users: List<User> - список усіх користувачів
- tracks: List<Track> - список усіх треків у системі

Операції:

- registerUser(u: User): зареєструвати нового користувача
- removeUser(u: User): видалити користувача з системи



Діаграма класів



Діаграма бази даних

3 варіанти використання та сценарії використання:

Сценарій 1: Відтворити трек

Передумови:

- Користувач відкрив програму.
- Аудіофайл було обрано через функцію «Відкрити файл».

Постумови:

- Починається відтворення обраного треку.

Взаємодіючі сторони:

- Користувач, клас Player.

Короткий опис:

- Користувач обирає трек і запускає його відтворення.

Основний перебіг подій:

- Користувач відкриває програму.
- Обирає аудіофайл через інтерфейс.
- Натискає кнопку «Відтворити».
- Система починає відтворювати файл.

Винятки:

- Якщо файл не обрано - виводиться повідомлення про помилку.
- Якщо формат файлу не підтримується - відтворення не починається.

Примітки:

- Функція «Відтворити» доступна лише після завантаження файлу.

Сценарій 2: Додати трек (Адмін)

Передумови:

- Адмін авторизувався в системі як адміністратор.

Постумови:

- Новий трек додано до загального списку треків.

Взаємодіючі сторони:

- Адмін, клас Admin, клас Track.

Короткий опис:

- Адміністратор додає новий трек через спеціальну форму.

Основний перебіг подій:

- Адмін входить до системи.
- Обирає функцію «Додати трек».
- Заповнює форму (назва, виконавець, тривалість).
- Натискає кнопку «Зберегти».
- Система створює новий об'єкт Track та зберігає його.

Винятки:

- Якщо поле не заповнене - виводиться повідомлення про помилку.
- Якщо сталася системна помилка - трек не зберігається.

Примітки:

- Додавати треки може лише адміністратор.

Сценарій 3: Перемотати трек вперед

Передумови:

- Трек вже відтворюється.
- Інтерфейс активний.

Постумови:

- Відтворення продовжується з нової позиції в аудіофайлі.

Взаємодіючі сторони:

- Користувач, клас Player.

Короткий опис:

- Користувач перемотує трек вперед для пропуску певної частини.

Основний перебіг подій:

- Користувач слухає трек.
- Натискає кнопку «Перемотати вперед».
- Система змінює позицію відтворення на декілька секунд уперед.
- Відтворення продовжується з нової точки.

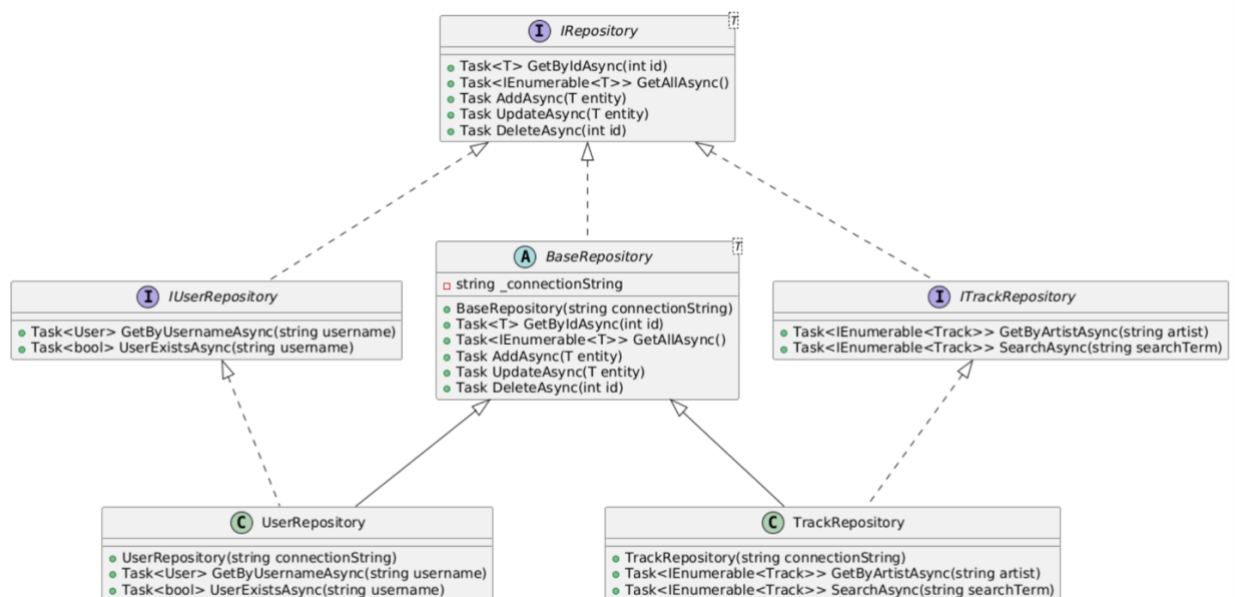
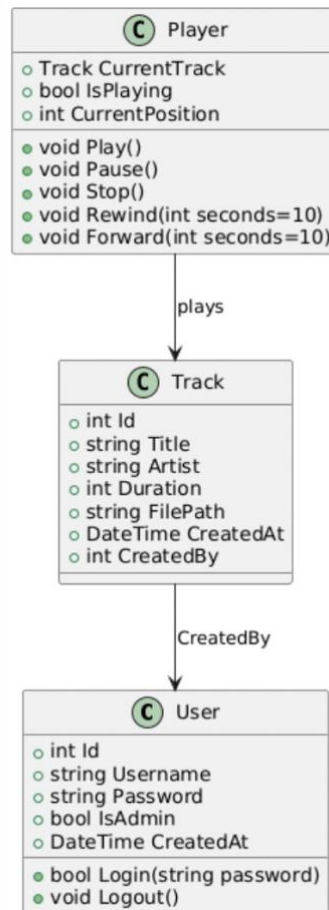
Винятки:

- Якщо перемотування перевищує довжину треку – система зупиняє відтворення.

Примітки:

- Перемотування працює лише під час активного відтворення.

На основі спроектованої діаграми класів предметної області розробити основні класи та структуру бази даних системи. Класи даних повинні реалізувати шаблон Repository для взаємодії з базою даних:



Код реалізованих класів:

```
0 references
1  ✓ public class Player
2  {
3      3 references
4      public Track CurrentTrack { get; set; }
5      3 references
6      public bool IsPlaying { get; set; }
7      5 references
8      public int CurrentPosition { get; set; }
9
10     0 references
11     public void Play()
12     {
13         if (CurrentTrack != null)
14         {
15             IsPlaying = true;
16             // Логіка відтворення
17         }
18     }
19
20     0 references
21     public void Pause()
22     {
23         IsPlaying = false;
24     }
25
26     0 references
27     public void Stop()
28     {
29         IsPlaying = false;
30         CurrentPosition = 0;
31     }
32
33     0 references
34     public void Rewind(int seconds = 10)
35     {
36         CurrentPosition = Math.Max(0, CurrentPosition - seconds);
37     }
38
39     0 references
40     public void Forward(int seconds = 10)
41     {
42         if (CurrentTrack != null)
43         {
44             CurrentPosition = Math.Min(CurrentTrack.Duration, CurrentPosition + seconds);
45         }
46     }
47 }
```

Player.cs (Models)

```
0 references
1  public class Track
2  {
3      0 references
4      public int Id { get; set; }
5      0 references
6      public string Title { get; set; }
7      0 references
8      public string Artist { get; set; }
9      0 references
10     public int Duration { get; set; }
11     0 references
12     public string FilePath { get; set; }
13     0 references
14     public DateTime CreatedAt { get; set; }
15     0 references
16     public int CreatedBy { get; set; }
17 }
```

Track.cs(Models)

```

0 references
1 public class User
2 {
3     0 references
4     public int Id { get; set; }
5     0 references
6     public string Username { get; set; }
7     1 reference
8     public string Password { get; set; }
9     0 references
10    public bool IsAdmin { get; set; }
11    0 references
12    public DateTime CreatedAt { get; set; }
13
14    0 references
15    public bool Login(string password)
16    {
17        return Password == password;
18    }
19
20    0 references
21    public void Logout()
22    {
23    }
24 }

```

User.cs(Models)

```

1 reference
1 public abstract class BaseRepository<T> : IRepository<T> where T : class
2 {
3     1 reference
4     protected readonly string _connectionString;
5
6     0 references
7     protected BaseRepository(string connectionString)
8     {
9         _connectionString = connectionString;
10    }
11
12    0 references
13    public abstract Task<T> GetByIdAsync(int id);
14    0 references
15    public abstract Task<IEnumerable<T>> GetAllAsync();
16    0 references
17    public abstract Task AddAsync(T entity);
18    0 references
19    public abstract Task UpdateAsync(T entity);
20    0 references
21    public abstract Task DeleteAsync(int id);
22 }

```

BaseRepository(Repositories.Implementations)

```

    public class TrackRepository : BaseRepository<Track>, ITrackRepository
    {
        0 references
        public TrackRepository(string connectionString) : base(connectionString) { }

        0 references
        public override async Task<Track> GetByIdAsync(int id)
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = "SELECT * FROM Tracks WHERE Id = @Id";
            return await connection.QueryFirstOrDefaultAsync<Track>(sql, new { Id = id });
        }

        0 references
        public override async Task<IEnumerable<Track>> GetAllAsync()
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = "SELECT * FROM Tracks ORDER BY Title";
            return await connection.QueryAsync<Track>(sql);
        }

        0 references
        public override async Task AddAsync(Track track)
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = @"INSERT INTO Tracks (Title, Artist, Duration, FilePath, CreatedBy)
                        VALUES (@Title, @Artist, @Duration, @FilePath, @CreatedBy);
                        SELECT CAST(SCOPE_IDENTITY() as int)";
            track.Id = await connection.ExecuteScalarAsync<int>(sql, track);
        }

        0 references
        public override async Task UpdateAsync(Track track)
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = @"UPDATE Tracks SET Title = @Title, Artist = @Artist,
                        Duration = @Duration, FilePath = @FilePath WHERE Id = @Id";
            await connection.ExecuteAsync(sql, track);
        }

        0 references
        public override async Task DeleteAsync(int id)
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = "DELETE FROM Tracks WHERE Id = @Id";
            await connection.ExecuteAsync(sql, new { Id = id });
        }

        0 references
        public async Task<IEnumerable<Track>> GetByArtistAsync(string artist)
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = "SELECT * FROM Tracks WHERE Artist LIKE @Artist ORDER BY Title";
            return await connection.QueryAsync<Track>(sql, new { Artist = $"%{artist}%" });
        }

        0 references
        public async Task<IEnumerable<Track>> SearchAsync(string searchTerm)
        {
            using var connection = new SqlConnection(_connectionString);
            var sql = @"SELECT * FROM Tracks
                        WHERE Title LIKE @SearchTerm OR Artist LIKE @SearchTerm
                        ORDER BY Title";
            return await connection.QueryAsync<Track>(sql, new { SearchTerm = $"%{searchTerm}%" });
        }
    }

```

TrackRepository.cs(Repositories.Implementations)

```

1 reference
public class UserRepository : BaseRepository<User>, IUserRepository
{
    0 references
    public UserRepository(string connectionString) : base(connectionString) { }

    0 references
    public override async Task<User> GetByIdAsync(int id)
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = "SELECT * FROM Users WHERE Id = @Id";
        return await connection.QueryFirstOrDefaultAsync<User>(sql, new { Id = id });
    }

    0 references
    public override async Task<IEnumerable<User>> GetAllAsync()
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = "SELECT * FROM Users";
        return await connection.QueryAsync<User>(sql);
    }

    0 references
    public override async Task AddAsync(User user)
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = @"INSERT INTO Users (Username, Password, IsAdmin)
VALUES (@Username, @Password, @IsAdmin);
SELECT CAST(SCOPE_IDENTITY() as int)";
        user.Id = await connection.ExecuteScalarAsync<int>(sql, user);
    }

    0 references
    public override async Task UpdateAsync(User user)
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = @"UPDATE Users SET Username = @Username, Password = @Password,
IsAdmin = @IsAdmin WHERE Id = @Id";
        await connection.ExecuteAsync(sql, user);
    }

    0 references
    public override async Task DeleteAsync(int id)
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = "DELETE FROM Users WHERE Id = @Id";
        await connection.ExecuteAsync(sql, new { Id = id });
    }

    0 references
    public async Task<User> GetByUsernameAsync(string username)
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = "SELECT * FROM Users WHERE Username = @Username";
        return await connection.QueryFirstOrDefaultAsync<User>(sql, new { Username = username });
    }

    0 references
    public async Task<bool> UserExistsAsync(string username)
    {
        using var connection = new SqlConnection(_connectionString);
        var sql = "SELECT COUNT(1) FROM Users WHERE Username = @Username";
        var count = await connection.ExecuteScalarAsync<int>(sql, new { Username = username });
        return count > 0;
    }
}

```

UserRepository.cs(Repositories.Implementations)

```

0 references
public interface IRepository<T> where T : class
{
    0 references
    Task<T> GetByIdAsync(int id);
    0 references
    Task<IEnumerable<T>> GetAllAsync();
    0 references
    Task AddAsync(T entity);
    0 references
    Task UpdateAsync(T entity);
    0 references
    Task DeleteAsync(int id);
}

```

IRepository(Repositories.Interfaces)

```

0 references
public interface ITrackRepository : IRepository<Track>
{
    0 references
    Task<IEnumerable<Track>> GetByArtistAsync(string artist);
    0 references
    Task<IEnumerable<Track>> SearchAsync(string searchTerm);
}

```

ITrackRepository(Repositories.Interfaces)

```

0 references
public interface IUserRepository : IRepository<User>
{
    0 references
    Task<User> GetByUsernameAsync(string username);
    0 references
    Task<bool> UserExistsAsync(string username);
}

```

IUserRepository(Repositories.Interfaces)

Висновок:

У межах лабораторної роботи я навчилася створювати діаграми варіантів використання та діаграми класів, розробляти сценарії використання, а також застосовувати шаблон Repository для взаємодії з базою даних.